

Project Report: Library Manager Spring Boot Application

GitHub URL

<https://github.com/bhanage-viraj/library-manager-assignment>

Introduction

The Library Manager application is a Spring Boot web application built to manage two related entities: `Author` and `Book`. The application supports creating, reading, and updating records through JSP pages. It uses Spring MVC, Spring Data JPA, Hibernate, H2 database, JSTL, and CSS for styling.

Entity Relationship Design

The application uses two entities: `Author` and `Book`.

`Author` stores:

- id
- name
- email
- country
- biography

`Book` stores:

- id
- title
- isbn
- genre
- publication year
- price
- author reference

The relationship is one-to-many: one author can have many books, and each book belongs to one author.

```
```java
@OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Book> books;
```

```
```java
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "author_id", nullable = false)
private Author author;
```

Database Population

The database tables are generated automatically using JPA annotations and Hibernate. The application uses an H2 in-memory database. Sample data is inserted using `data.sql`, which contains 10 authors and 10 books.

Create Operation

The create operation is implemented using JSP forms.

Author creation:

- URL: `/authors/new`

- Controller method: `AuthorController#createAuthor`

Book creation:

- URL: `/books/new`
- Controller method: `BookController#createBook`

The form data is bound using `@ModelAttribute` and validated using `@Valid`. Duplicate email and duplicate ISBN errors are handled using `DataIntegrityViolationException`.

Read Operation

The read operation displays all saved records in JSP tables.

Author list:

- URL: `/authors`
- Controller method: `AuthorController#listAuthors`

Book list:

- URL: `/books`
- Controller method: `BookController#listBooks`

A custom repository query performs an inner join between `Book` and `Author`.

```
```java
@Query("""
 select new com.example.librarymanager.dto.BookAuthorView(
 b.id, b.title, b.isbn, b.genre, b.publicationYear,
 b.price, a.id, a.name, a.email
)
 from Book b
 inner join b.author a
 order by a.name asc, b.title asc
""")
List<BookAuthorView> findBookAuthorViews();
```
```

Update Operation

The update operation allows editing existing authors and books.

Author update:

- URL: `/authors/{id}/edit`
- Controller method: `AuthorController#updateAuthor`

Book update:

- URL: `/books/{id}/edit`
- Controller method: `BookController#updateBook`

The service layer first fetches the existing entity, updates editable fields, and saves the result back to the database.

Repository Layer

The repositories extend `JpaRepository`.

```
```java
```

```
public interface AuthorRepository extends JpaRepository<Author, Long>
```

```
```java
public interface BookRepository extends JpaRepository<Book, Long>
```

Custom methods include:

- `findByEmail`
- `findByIsbn`
- `findBookAuthorViews`

Service Layer

The service layer contains business logic and connects controllers with repositories. `BookService` resolves the selected author before saving a book, ensuring that every book has a valid author relationship.

View Layer

The JSP pages are stored in `src/main/webapp/WEB-INF/jsp`.

Pages created:

- `authors/list.jsp`
- `authors/form.jsp`
- `books/list.jsp`
- `books/form.jsp`
- `fragments/navigation.jspf`

CSS styling is implemented in `src/main/resources/static/css/styles.css`.

Testing

JUnit and Mockito tests were added for repository and service layers.

Repository tests verify:

- seeded data loading
- custom finder methods
- inner join query result

Service tests verify:

- create/save behavior
- update behavior
- missing author handling
- repository integration

Run tests using:

```
```bash
mvn test
```
```

Screenshots To Include

Add screenshots of:

- Books list page
- Add book page

- Authors list page
- Add author page
- Update book page
- Update author page

Challenges Faced

One challenge was handling the relationship between books and authors in the book form. The form submits an author id, but JPA needs a managed `Author` entity. This was solved in the service layer by looking up the selected author before saving the book.

Another challenge was displaying user-friendly validation errors for duplicate values. This was handled by catching integrity exceptions and binding clear error messages back to the JSP form fields.

Conclusion

The Library Manager application satisfies the assignment requirements by implementing two related JPA entities, database population, create/read/update operations, JSP views, service and repository layers, custom inner join query, CSS styling, and unit tests.